

Instruments Playground SDK

快速入门

Ver. 2025.07.26

目录

Instruments Playground SDK 快速入门	1
编写一个控制示波器和信号源的“波形验证”程序	2
1. 硬件配置	2
2. 安装 IP SDK	2
3. 在 Windows 系统下使用 SDK	3
3.1 使用 SDK 的 Python API	3
3.1.1 环境要求	3
3.1.2 环境搭建	3
Python 示例代码	9
3.2 使用 SDK 的 C/C++ API	13
C/C++ 示例代码	19
3.3 使用 SDK 的 MATLAB API	22
MATLAB 示例代码	24
4. 在树莓派的 Raspberry Pi OS 下使用 SDK	26
4.1 准备工作	26
4.2 使用 SDK 的 Python API	27
4.3 使用 SDK 的 C/C++ API	29

编写一个控制示波器和信号源的“波形验证”程序

本教程将指导您在 Windows 或树莓派 OS 上编写并运行一个 IP-SDK 示例程序：控制信号源输出 4V/1kHz 正弦波，然后通过示波器获取该波形数据。

1. 硬件配置

1. 将雨珠设备通过 USB 线连接到 PC 或树莓派；
2. 将示波器的通道 1 与信号源的通道 1 连接在一起，连接方法如下：
 - **雨珠 2/雨珠 3**：示波器和信号源均为排针，使用杜邦线按以下方式连接：
 - 示波器的“1+”端口接信号源的“W1”端口；
 - 示波器的“1-”端口接地（↓ 或 GND）；
 - **雨珠 X**：示波器和信号源均为 BNC 接口
 - 使用 BNC 线将 **AWG1** 端口与 **OSC1** 端口连接在一起。
 - **雨珠 S**：示波器和信号源同时具有排针和 BNC 接口，可按上述方式中的一种连接。另外需要注意的是，雨珠 S 的示波器有一个物理开关控制输入信号是从 BNC 接口还是排针接入，使用时需要将该开关拨到合适的挡位。

2. 安装 IP-SDK

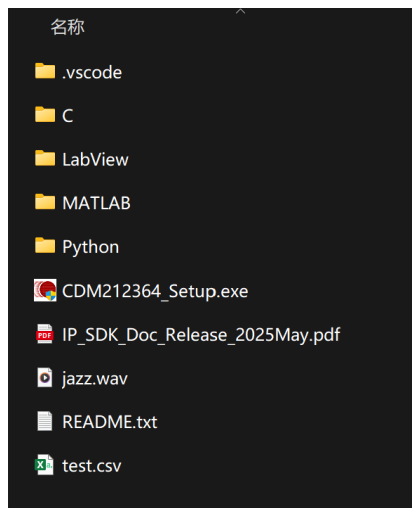
请下载最新版本的 IP-SDK，然后将 **IP-SDK 文件夹**放置在适当的路径下：

- **Windows**：推荐放置在 C:\YZKJ\下；
- **树莓派 OS**：推荐放置在 /home/[用户名]/下。

然后安装驱动：

- **Windows**：如果已经安装了仪器操场（IP），就不需要再安装驱动了；否则需要运行“CDM212364_Setup.exe”来安装驱动。**注意**：对于 32 位的 Windows 系统，即使安装了 IP，可能也需要手动安装驱动。
- **树莓派 OS**：按 4.1 准备工作进行操作。

IP-SDK 文件夹中通常包含以下文件：



3. 在 Windows 系统下使用 SDK

3.1 使用 SDK 的 Python API

3.1.1 环境要求

1. Python 3.x (经过 Python 3.13 测试)
2. Python 库: numpy, matplotlib

3.1.2 环境搭建

Python 环境搭建可以选择以下 2 种方法之一:

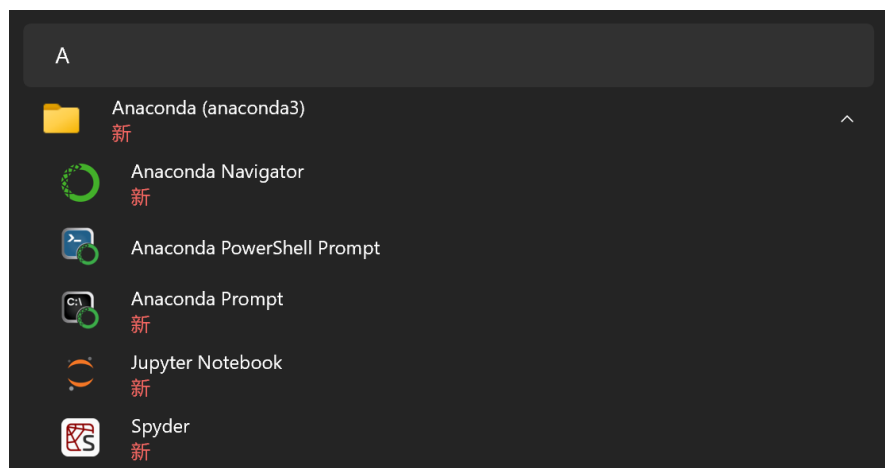
- 选项一: Anaconda 环境搭建 (推荐)
- 选项二: 轻量化的 Python 环境搭建

3.1.2.1 选项一: Anaconda 环境搭建 (推荐)

Anaconda 是一个用于科学计算的 Python 发行版本, 集成了大量常用的数据科学工具和库, 并提供了一个强大的包管理和环境管理工具 (conda), 使得安装、管理和部署软件包变得非常方便。

I. 安装 Anaconda

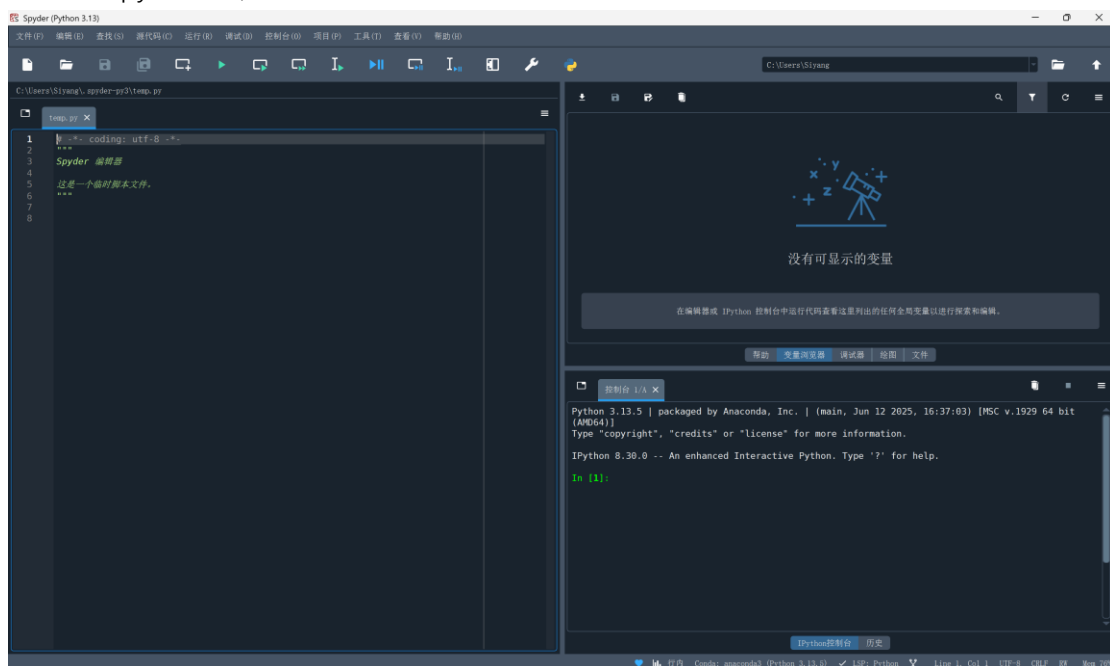
1. 下载 Anaconda Distribution Installers:
前往地址: <https://www.anaconda.com/download/success>, 并选择适合你的操作系统的版本;
2. 双击下载的.exe 文件 (如 Anaconda3-2025.06-0-Windows-x86_64.exe), 运行安装程序;
3. 安装过程中, 推荐勾选以下选项:
 - Register Anaconda3 as the system Python 3.x
 - Clear the package cache upon completion
4. 安装完成后, 可以在开始菜单中找到以下新安装的程序:



- **Anaconda Navigator:** 图形化界面 (GUI) 工具, 用于方便地管理 Python 环境和安装包以及启动数据科学应用, 而无需记忆命令行操作;
- **Anaconda Prompt:** 集成了命令提示符 (CMD) 的命令行工具, 已经预先配置好了 Conda 环境变量, 可以直接使用 conda、python、pip 等命令;
- **Anaconda PowerShell Prompt:** 集成了 Powershell 的命令行工具, 已经预先配置好了 Conda 环境变量, 可以直接使用 conda、python、pip 等命令;
- **Jupyter Notebook:** 交互式编程环境, 以单元格 (cell) 的形式组织代码、文本、公式和可视化结果, 名称源自 **Julia + Python + R** (其所支持的语言);
- **Spyder:** 专为科学计算设计的 Python IDE, 集成了代码编辑、调试、变量查看、交互式控制台等功能, 界面类似 MATLAB, 名称源自 **Scientific Python Development Environment**。

II. 运行示例代码

1. 打开 Spyder IDE, 初次运行界面如下:

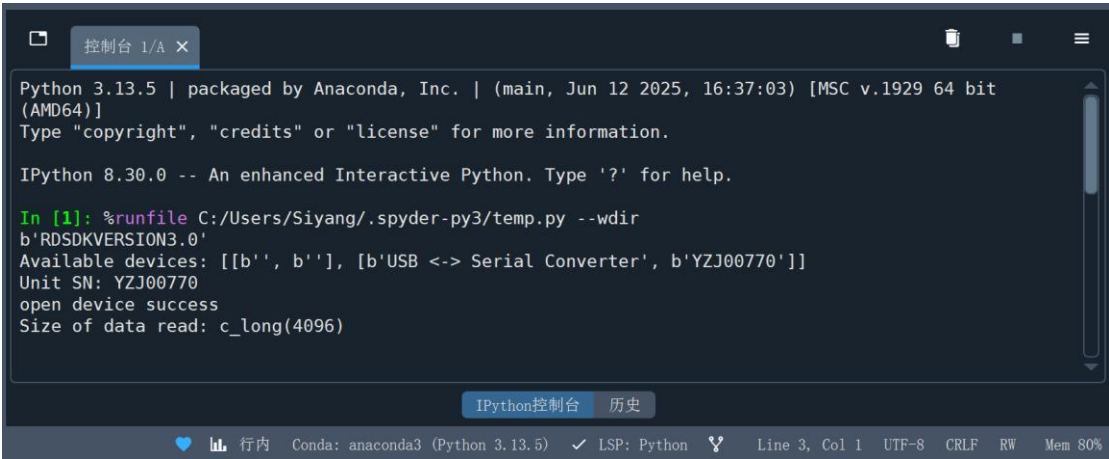


2. 删除左侧编辑器中已有的文字, 将**示例代码**拷贝到编辑器中。拷贝时, 需要注意将以下代码中的路径更换为你放置 IP-SDK 的路径;

```
import sys
# 将 IP-SDK (Python) 添加到系统环境变量 PATH 中
sys.path.append('C:\YZKJ\IP-SDK\Python\src')
```

3. 点击运行文件 (绿色箭头) 或按 F5;

4. 如果运行成功，右下方的控制台应有如下显示：

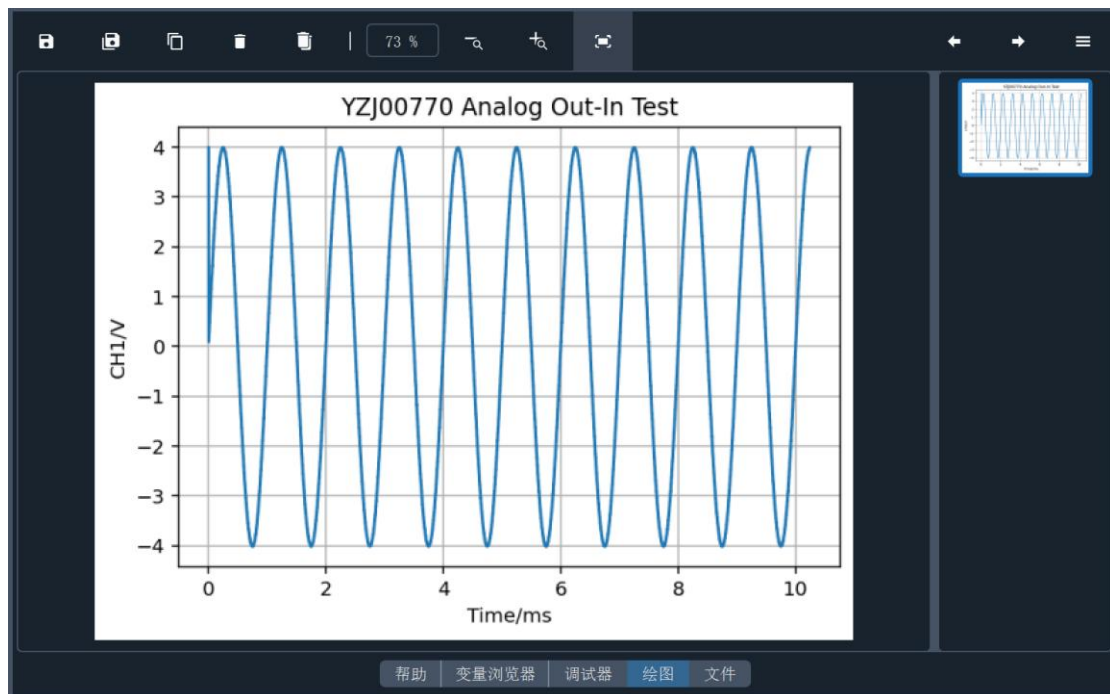


```
Python 3.13.5 | packaged by Anaconda, Inc. | (main, Jun 12 2025, 16:37:03) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.30.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: %runfile C:/Users/Siyang/.spyder-py3/temp.py --wdir
b'ROSDKVERSION3.0'
Available devices: [[b'', b''], [b'USB <-> Serial Converter', b'YZJ00770']]
Unit SN: YZJ00770
open device success
Size of data read: c_long(4096)
```

5. 在右上方的“绘图”选项中可以查看绘制的波形：



6. 上述代码的编辑与运行发生在 temp.py 文件中，这个文件由 Spyder 自动生成，且只能保存在其工作目录中。如果你想将代码保存在一个单独的文件中，可以点击新建文件 (Ctrl+N)，然后再编辑代码。

III. 运行失败时如何排查问题

1. 首先检查硬件连接是否有问题：

可以打开带有图形界面 (GUI) 的仪器操场 (IP)，并运行示波器以及信号源 CH1，检验示波器是否可以观察到信号源输出的波形。

2. 如果硬件连接没有问题，分段运行代码以确定问题出现在哪里。在 Spyder 中，以下工具都适用于分段运行代码：

- IPython 控制台；
- 运行选定的代码或当前行 (F9) ；

- 运行单元格 (Ctrl+Return)，单元格是通过特殊注释标记的代码块。定义单元格的方式为在代码中插入 `###`，后续代码会被视为一个独立单元格，比如：

```
### 单元格 1
print("这是第一个单元格")
x = 10

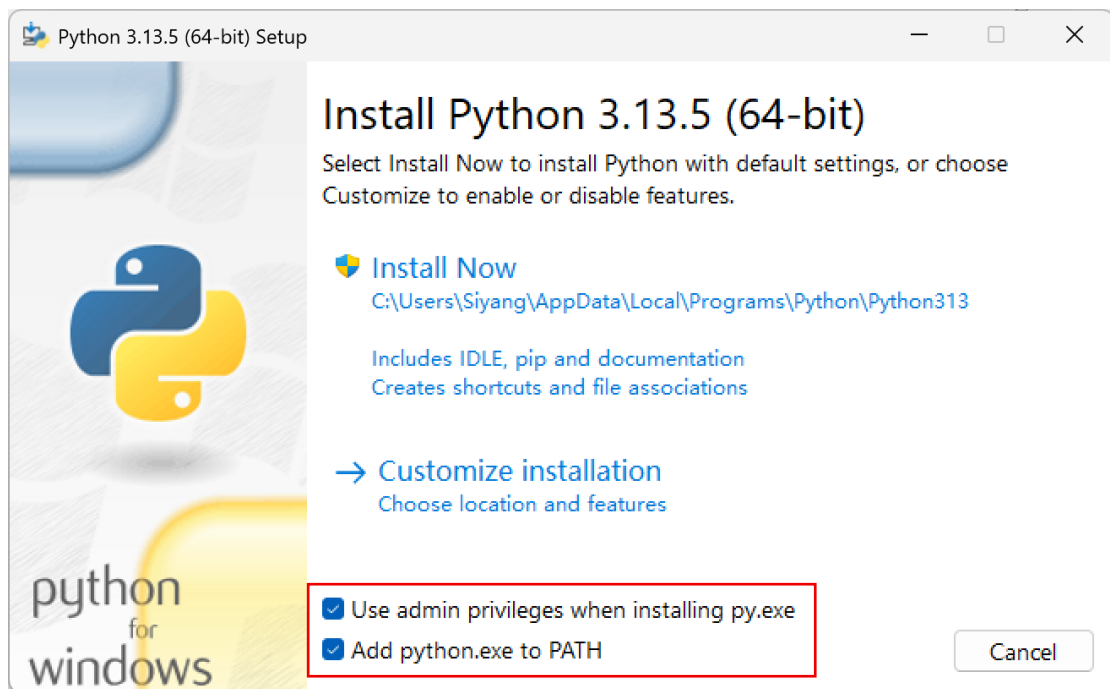
### 单元格 2
print("这是第二个单元格")
y = x + 5 # 可以访问上一个单元格的变量
```

除上述方法外，如果你了解如何调试代码，也可以选择“调试文件 (Ctrl+F5)”。

3.1.2.2 选项二：轻量化的 Python 环境搭建

I. 安装 Python

前往 <https://www.python.org/downloads/> 下载 Python 的最新稳定版本，并安装；安装时，推荐勾选以下选项，然后点击 **Install Now** 安装。



启动命令行工具（命令提示符 CMD 或 Windows PowerShell 均可），然后输入以下指令验证安装是否成功，如果显示了 Python 版本号，则安装成功。

```
> python --version
Python 3.13.5
```

* 在关于命令行的代码中，可能会使用 `>` 或 `$` 代表命令提示符，提示符后为需要输入的指令，没有提示符的行是指令运行后输出的结果。

II. 安装库

1. 更换 pip 软件源（可选）

如果默认源的网络连接较差，更换源可以提高下载速度，请按说明 <https://mirrors.tuna.tsinghua.edu.cn/help/pypi/> 操作；

2. 在命令行输入以下指令安装 numpy 和 matplotlib 库：

```
> pip install numpy matplotlib
```

II. 运行示例代码

1. 打开一种文本或代码编辑器(比如 Windows 预装的记事本)，将示例代码拷贝到编辑器中并保存，保存的文件命名为 **test.py**，保存的路径为 **C:\Users\[用户名]**（此为命令行工具启动时的默认路径）；

2. 在命令行输入以下命令运行代码：

```
> python test.py
```

如果你的代码文件没有放在默认路径下，需要前往该路径：

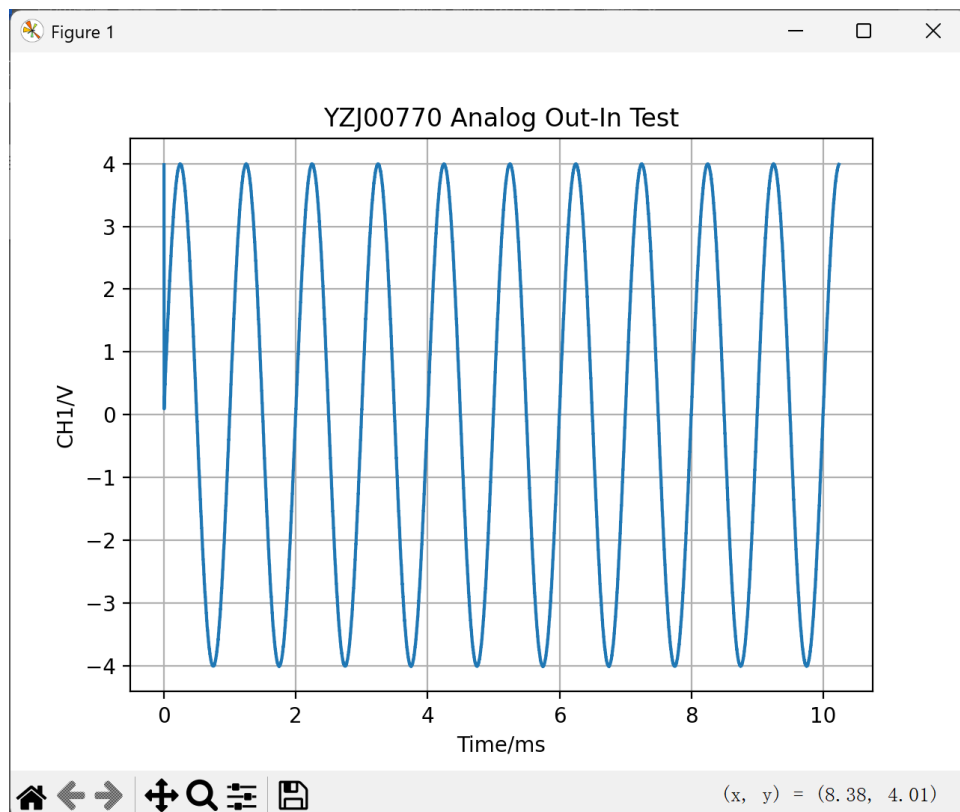
```
> cd C:\Users\[用户名]
```

或者也可以使用代码文件的绝对路径：

```
> python C:\Users\[用户名]
```

3. 代码运行成功后，应显示如下结果

- 弹出的 matplotlib 绘图窗口：



- 命令行输出结果:

```
b'RDSDKVERSION3.0'
Available devices: [[b'USB <-> Serial Converter', b'YZJ00770']]
Unit SN: YZJ00770
open device success
Size of data read: c_long(4096)
```

最后, 如果你准备长期工作在 Windows 的命令行环境下, 推荐安装以下工具:

- Windows Terminal
- PowerShell 7

另外, 也推荐安装一个代码编辑器, 比如:

- Visual Studio Code

这些都可以在 Microsoft Store 中找到。

Python 示例代码

```
from dataclasses import dataclass
import time

import numpy as np
import matplotlib.pyplot as plt

import sys
# 将 IP-SDK (Python) 添加到系统环境变量 PATH 中
sys.path.append('C:\YZKJ\IP-SDK\Python\src')
from pyRD import RD
from pyRD.core.RDconstant import *

# 创建 RD 类的实例
rd = RD()

# 枚举并打印可用的设备列表
rd.DeviceEnumLists()
print(f"Available devices: {rd.devicelist}")

# 查找序列号包含 'YZ' 的设备
usb_port = None
for i, device in enumerate(rd.devicelist):
    device_sn = device[1].decode()
    if 'YZ' in device_sn:
        usb_port = i

# 未找到设备报错
if usb_port is None:
    raise(RuntimeError("Device not found."))

# 获取匹配设备的序列号
device_sn = rd.devicelist[usb_port][1].decode()
print(f"Unit SN: {device_sn}")

# 打开设备
status = rd.DeviceOpen(usb_port)

"""示波器配置"""
# 定义示波器配置参数的数据类
@dataclass
class AnalogInParams:
```

```
ch: int
range: int
freq: float
trig_timeout: float
trig_src: int
trig_type: int
trig_slope: int
trig_level: float
buffersize: int

# 定义应用示波器配置参数的函数
def AnalogInApplyConfig(rd, ai):
    rd.AnalogInCHEnable(ai.ch, True)
    rd.AnalogInCHRangeSet(ai.ch, ai.range)
    rd.AnalogInFrequencySet(ai.freq)
    rd.AnalogInTriggerAutoTimeoutSet(ai.trig_timeout)
    rd.AnalogInTriggerSourceSet(ai.trig_src)
    rd.AnalogInTriggerTypeSet(ai.trig_type)
    rd.AnalogInTriggerConditionSet(ai.trig_slope)
    rd.AnalogInTriggerLevelSet(ai.trig_level, ai.range)
    rd.AnalogInBufferSizeSet(ai.buffersize)

# 创建一套示波器的配置参数, 命名为 ai0
ai0 = AnalogInParams(
    ch = 0,                                # CH1
    range = 5,                             # 量程: 5V
    freq = 400e3,                           # 采样率: 400kHz
    trig_timeout = 0,                       # 自动触发时间: 永不自动触发
    trig_src = RDTRIGSRCDetectorAnalogInCH1, # 触发源: CH1
    trig_type = RDTRIGTYPEEdge,             # 触发方式: 边沿触发
    trig_slope = RDTriggerSlopeRise,        # 触发方向: 上升沿触发
    trig_level = 0,                         # 触发电平: 0V
    buffersize = 4096,                      # 缓冲区大小: 4096 字节
)

# 应用示波器配置 ai0
AnalogInApplyConfig(rd, ai0)

"""信号源配置"""
# 定义信号源配置参数的数据类
@dataclass
class AnalogOutParams:
    ch: int
    node: int
```

```
func: int
freq: float
amp: float
offset: float
symmetry: int
phase: int

# 定义应用信号源配置参数的函数
def AnalogOutApplyConfig(rd, ao):
    rd.AnalogOutNodeEnableSet(ao.ch, ao.node, True)
    rd.AnalogOutNodeFunctionSet(ao.ch, ao.node, ao.func)
    rd.AnalogOutNodeFrequencySet(ao.ch, ao.node, ao.freq)
    rd.AnalogOutNodeOffsetAmpSet(ao.ch, ao.node, ao.offset, ao.amp)
    rd.AnalogOutNodeSymmetrySet(ao.ch, ao.node, ao.symmetry)
    rd.AnalogOutNodePhaseSet(ao.ch, ao.node, ao.phase)

# 创建一套信号源的配置参数, 命名为 ao0
ao0 = AnalogOutParams(
    ch = 0,                                # CH1
    node = RDAnalogOutNodeCarrier,         # 节点: 载波节点
    func = RDFUNCSine,                     # 波形函数: 正弦波
    freq = 1000,                            # 频率: 1kHz
    offset = 0,                             # 偏执: 0V
    amp = 4,                                # 幅值: 4V
    symmetry = 50,                          # 对称性: 50
    phase = 0,                              # 相位: 0
)

# 应用信号源配置 ao0
AnalogOutApplyConfig(rd, ao0)

"""运行设备"""
# 启动信号源
rd.AnalogOutConfigure(ao0.ch, True)

# 等待 1 秒至输出稳定
time.sleep(1)

# 启动示波器
rd.AnalogInRun(True)

# 检查示波器状态是否就绪
success = False
for _ in range(10):                        # 最多尝试 10 次
```

```
rd.AnalogInStatus()          # 获取当前状态
if rd.analoginstatus == 2:    # 状态为 2 表示采集完成
    success = True
    break

# 信号采集失败报错
if not success:
    raise(RuntimeError('Scope reading failed.'))

# 读取采集到的数据
rd.AnalogInRead(ai0.bufferSize, ai0.ch)

# 打印读取到的数据点数
print(f"Size of data read: {rd.aibacksizech1}")

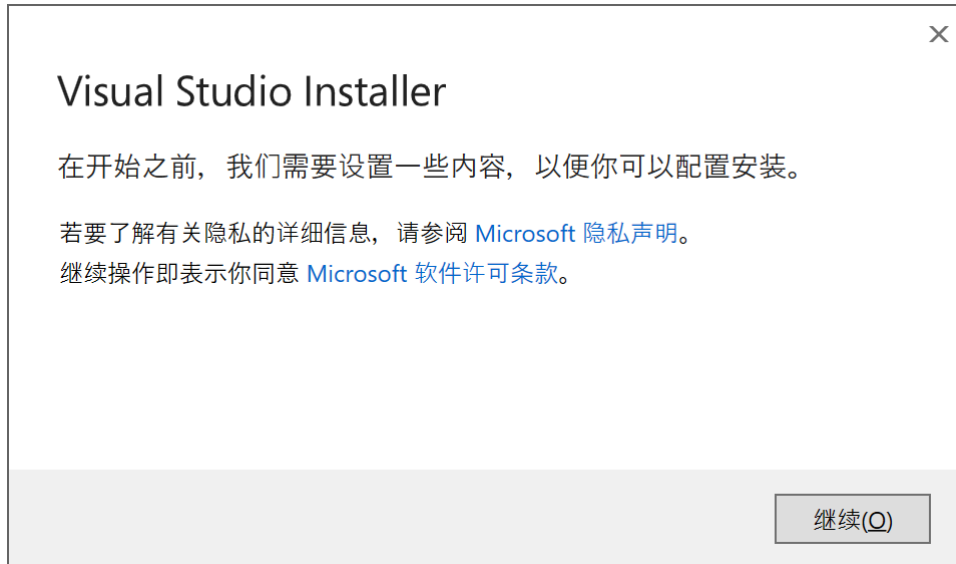
# 使用 matplotlib 绘制采集到的波形
x = np.linspace(0, 1 / ai0.freq * ai0.bufferSize * 1000,
ai0.bufferSize)
y = np.array(rd.aidatach1)
plt.plot(x, y)
plt.grid(True)
plt.xlabel("Time/ms")
plt.ylabel("CH1/V")
plt.title(f"{device_sn} Analog Out-In Test")
plt.show()

"""善后工作"""
# 关闭信号源
rd.AnalogOutConfigure(ao0.ch, False)
# 关闭示波器
rd.AnalogInRun(False)
# 关闭设备
rd.DeviceClose()
```

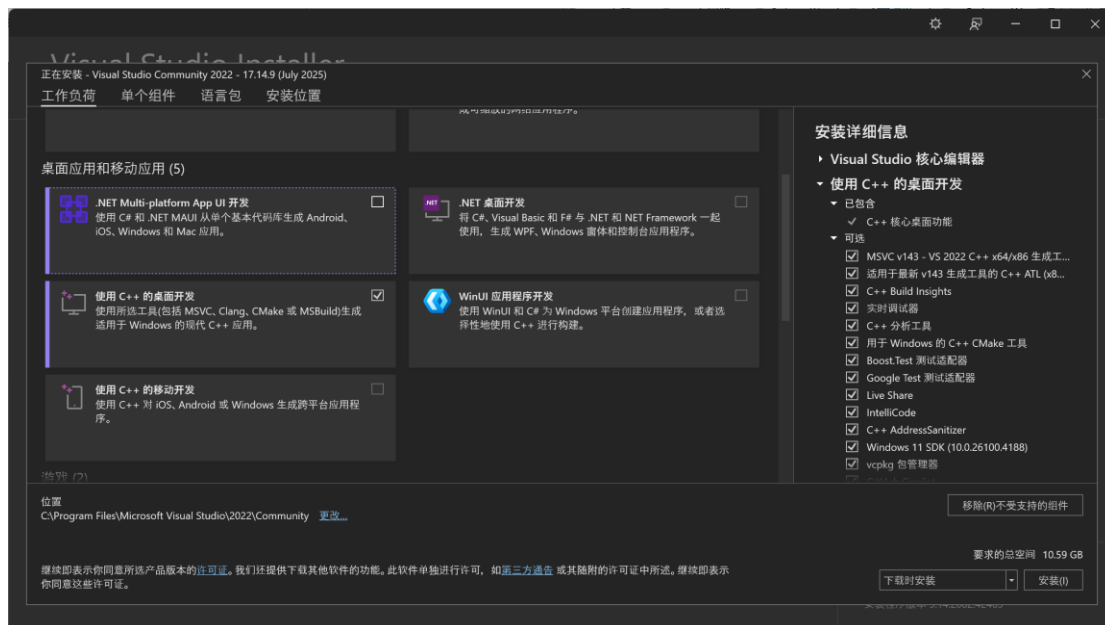
3.2 使用 SDK 的 C/C++ API

I. 安装 Visual Studio Community

1. 前往 <https://visualstudio.microsoft.com/zh-hans/vs/community/> 下载 VS;
2. 双击下载的文件 VisualStudioSetup.exe, 运行安装程序 Visual Studio Installer:



3. 在选择工作负荷的页面, 勾选**使用 C++ 的桌面开发**选项, 该工具包含我们所需要的 C++ 编译器 MSVC, 然后点击安装。



4. 安装可能需要较长时间, 等待安装完成。

II. 创建一个新 C++ 控制台项目

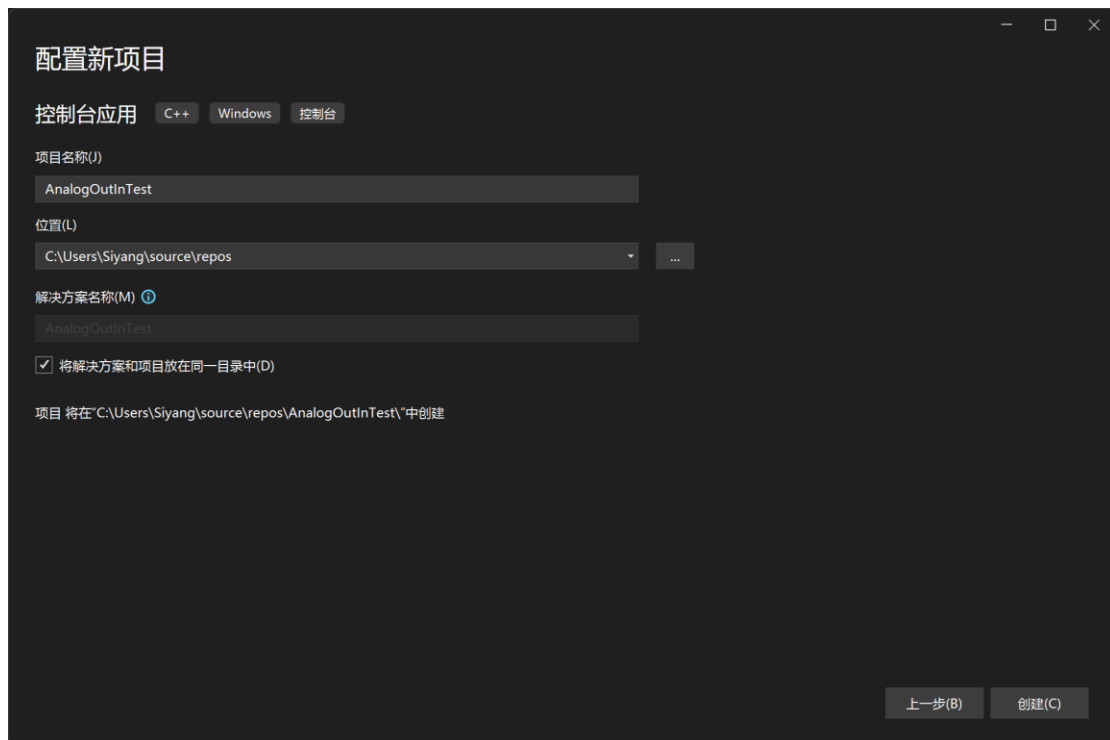
1. 启动 Visual Studio 2022，选择创建新项目；



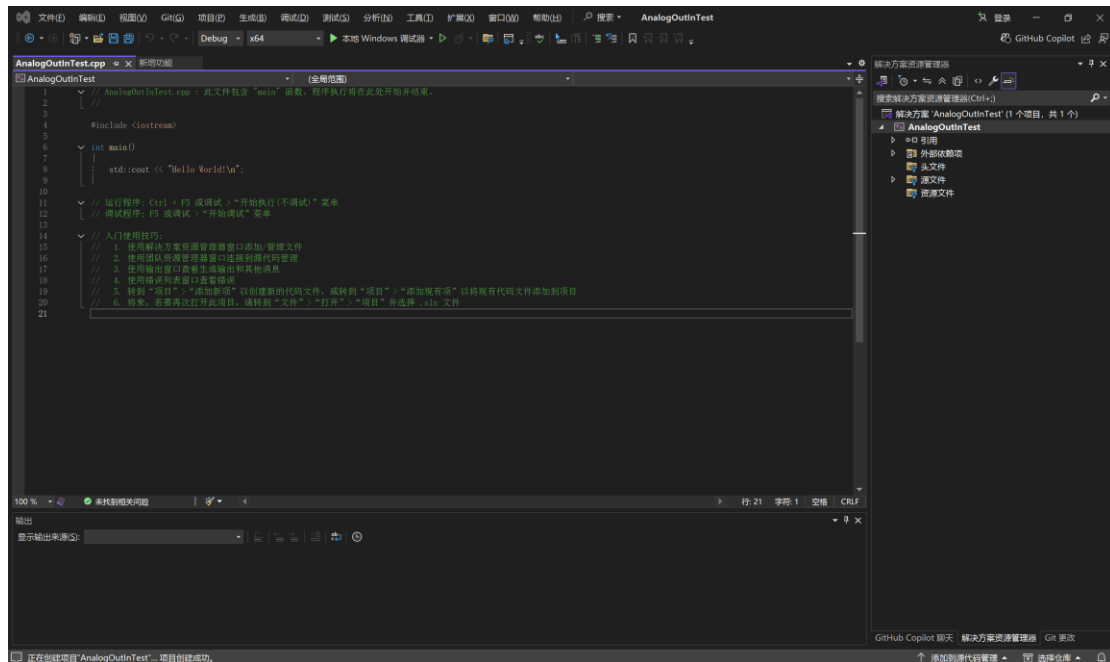
2. 在项目模板中选择控制台应用；



3. 输入项目名称和位置，推荐勾选将解决方案和项目放在同一目录中；



4. 创建成功后的项目界面如下所示：

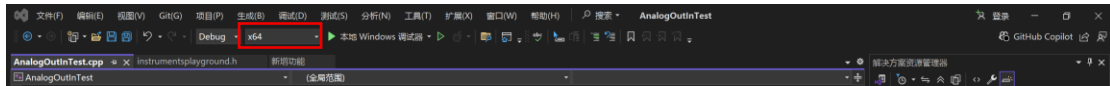


III. 使用 IP-SDK 的动态链接库 (.dll)

要在 VS 项目中使用 IP-SDK，需要准备以下文件：

- **DLL 文件：**
 - ftd2xx64.dll 或 ftd2xx32.dll
 - InstrumentsPlayground.dll
 - sqlite3.dll
- **导入库文件：**
 - InstrumentsPlayground.lib
- **头文件：**
 - ftd2xx.h
 - instrumentsplayground.h
 - sqlite\sqlite3.h

这些文件的一部分存放在 IP-SDK/C 中，另一部分有 64 位版本和 32 位版本的区分，分别存放在 IP-SDK/C/64 或 IP-SDK/C/32 中。具体使用哪个版本取决于你创建的项目的活动平台，可以在这里看到：



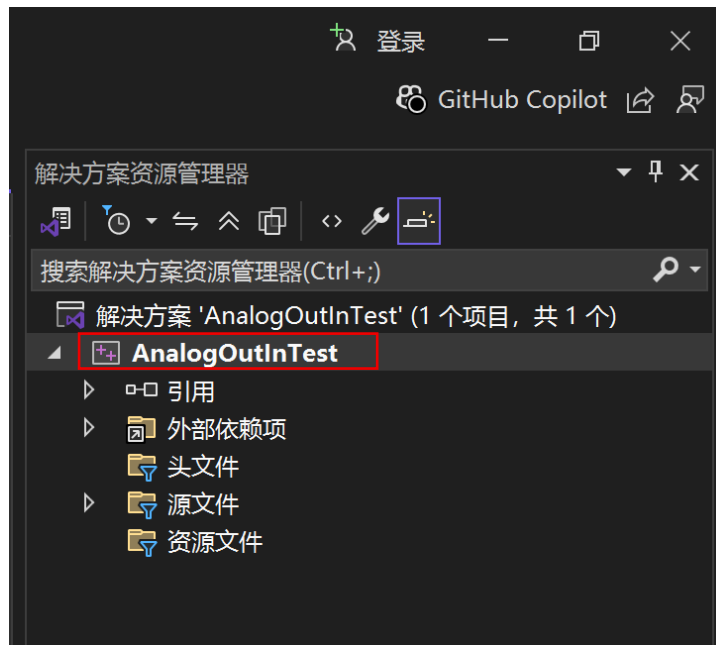
在 VS 项目中使用这些文件时，可以通过以下 2 种方式：

选项一：为 VS 添加文件路径，使其可以找到相应文件（推荐）；

选项二：将 .dll 文件放到程序输出目录（如 **Debug/** 或 **Release/**）；将 .lib 文件和 .h 文件放到与 **main** 函数所在的源文件（.cpp 文件）相同的目录下。

以下将详细说明**选项一**的操作方法：

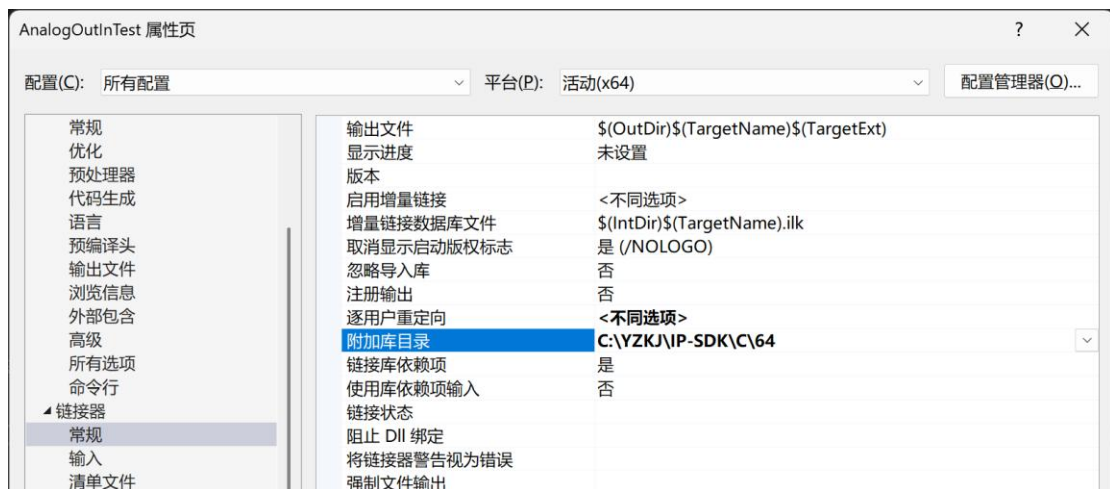
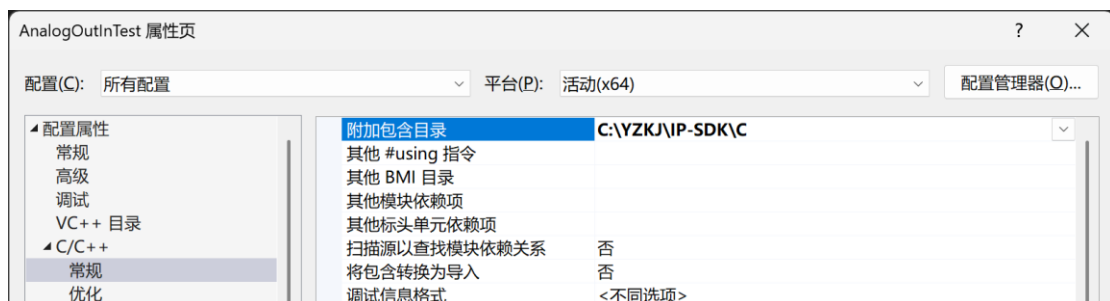
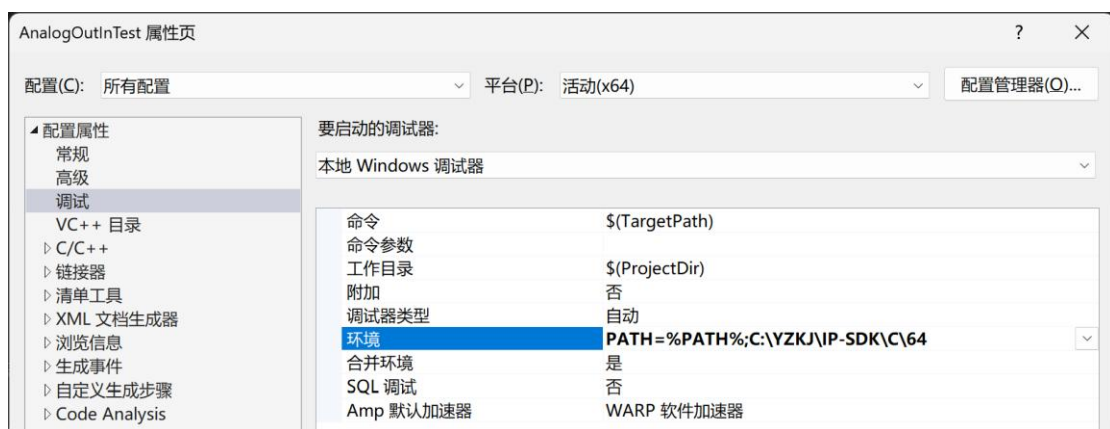
1. 右键点击项目名称（**AnalogOutInTest**），在弹出菜单中选择**属性**；

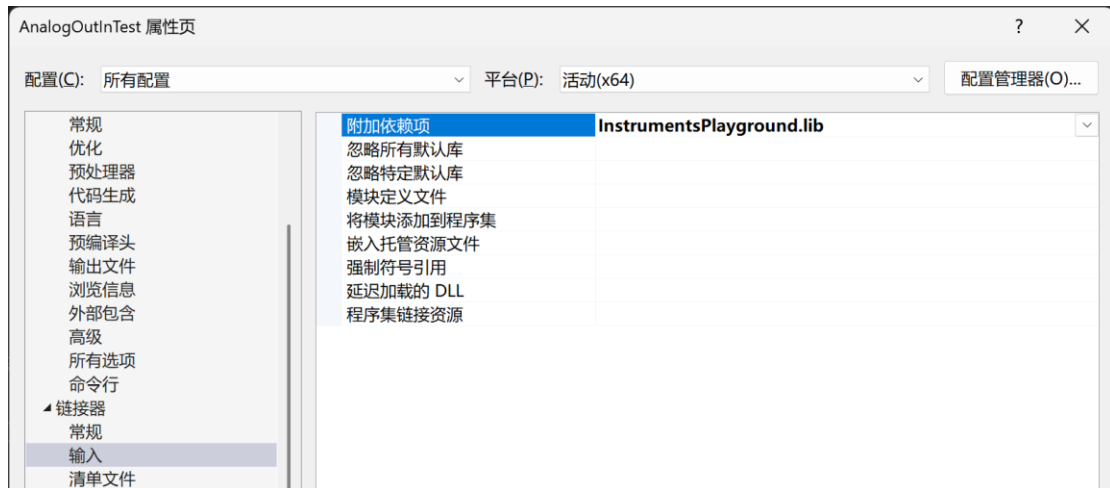


2. 属性页面上方的配置可以选择 Debug, Release 或所有配置, 推荐选择**所有配置**; 平台可以选择 Win32, x64 或所有平台, 推荐**保持默认选项**。然后在属性页面,

- 找到**调试→环境**, 在本项目中输入.dll 文件路径
 - PATH=%PATH%;C:\YZKJ\IP-SDK\C\64
- 找到**C/C++→常规→附加包含目录**, 在本项目中输入.h 文件的路径
 - C:\YZKJ\IP-SDK\C
- 找到**链接器→常规→附加包含目录**, 在本项目中输入.lib 文件的路径
 - C:\YZKJ\IP-SDK\C\64
- 找到**链接器→输入→附加依赖项**, 在本项目中输入.lib 文件的名称
 - InstrumentsPlayground.lib

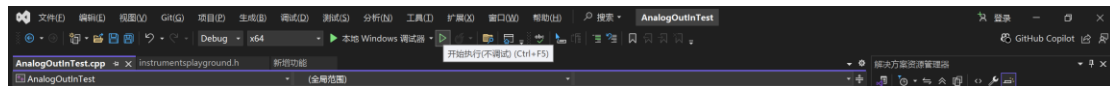
完成后的属性页面应如下所示:



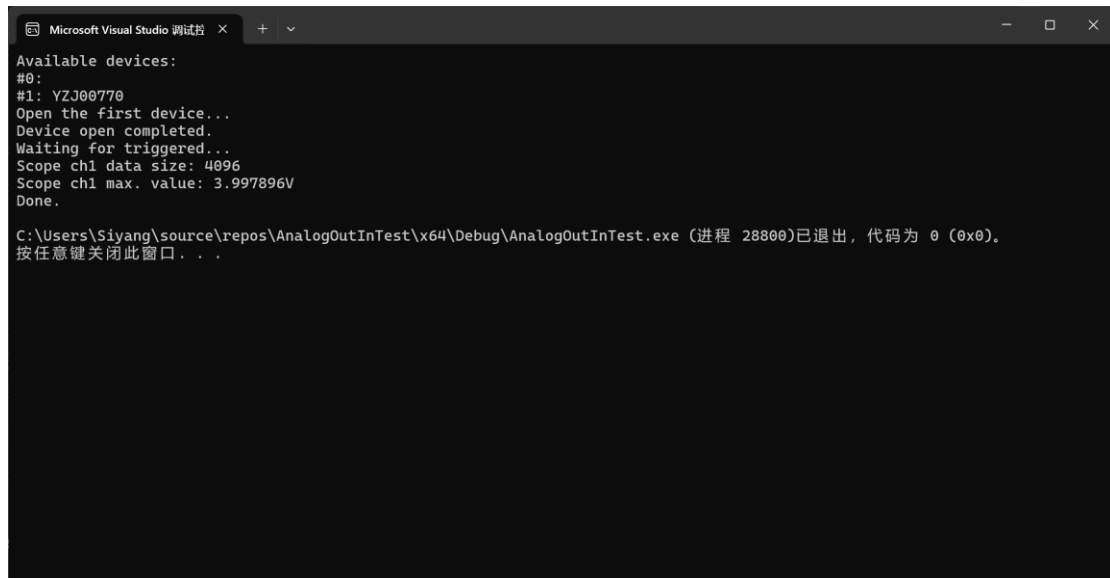


IV. 运行代码

1. 将示例代码拷贝到 AnalogOutInTest.cpp 文件中;
2. 点击开始执行（不调试）或按 **Ctrl+F5**;



3. 如果运行成功，在弹出的控制台窗口会显示如下结果：



C/C++示例代码

```
#include "instrumentsplayground.h"
#include <stdio.h>
#include <string.h>
#include <windows.h>
#include <algorithm>

int main(int carg, char** szarg) {
    FT_HANDLE ftHandle;           //设备句柄
    FT_STATUS status;             //设备返回状态
    DWORD devicecount;           //用于保存可用设备数量
    char Info[64], SN[64];        //可用设备的Info和SN
    bool device_selected = false; //是否选中了设备
    int device_no = 0;            //选中的设备的索引

    //查询可用设备数量
    RDEnumDeviceCount(&devicecount);

    //枚举所有可用设备并选中第1台包含“YZ”的设备
    printf("Available devices:\n");

    for (int i = 0; i < devicecount; i++) {
        RDEnumDeviceInfo(i, Info, SN, &ftHandle);
        printf("#%d: %s\n", i, SN);
        char* result = strstr(SN, "YZ");
        if (!device_selected && result != NULL) {
            device_selected = true;
            device_no = i;
        }
    }

    if (!device_selected) {
        printf("Device not found!\n");
        return 0;
    }

    //打开设备
    printf("Open the first device...\n");

    status = RDdeviceOpen(device_no, &ftHandle);
    if (status) {
        printf("Device open failed.\nStatus: %d\n", status);
    }
}
```

```
        return 0;
    }
    else {
        printf("Device open completed.\n");
    }

    //读取校准数组
    uchar calArr[50];
    RDCalibrationRead(ftHandle, calArr);

    //模拟输入设置
    int aiCh = 0;           //通道
    int cSamples = 4096;    //采样点数
    int range = 5;          //电压量程
    double trigLevel = 0.0; //触发电平
    RDState sts;            //采集状态
    double* rgdSamples;     //回读数组
    int backsize;           //回读点数

    //模拟输入通道1使能
    RDAnalogInChannelEnableSet(ftHandle, aiCh, true);
    //设置模拟输入的电压量程为5V
    RDAnalogInChannelRangeSet(ftHandle, aiCh, range);
    //设置模拟输入的采样频率频率为400kHz
    RDAnalogInFrequencySet(ftHandle, 400000);
    //设置模拟输入的缓冲区大小为采样点数
    rgdSamples = new double[cSamples];
    RDAnalogInBufferSizeSet(ftHandle, cSamples);
    //设置触发源：通道1
    RDAnalogInTriggerSourceSet(ftHandle, RDTRIGSRCDetectorAnalogInCH1);
    //不会自动触发
    RDAnalogInTriggerAutoTimeoutSet(ftHandle, 0);
    //触发方式：边沿触发
    RDAnalogInTriggerTypeSet(ftHandle, RDTRIGTYPEEdge);
    //触发电平：0V
    RDAnalogInTriggerLevelSet(ftHandle, trigLevel, range, aiCh, calArr);
    //上升沿触发
    RDAnalogInTriggerConditionSet(ftHandle, RDTriggerSlopeRise);

    //模拟输出设置
    int aoCh = 0;
    //模拟输出通道1使能
    RDAnalogOutNodeEnableSet(ftHandle, aoCh, RDAnalogOutNodeCarrier, true);
    //设置通道1的波形函数为正弦波
```

```
RDAnalogOutNodeFunctionSet(ftHandle, aoCh, RDAnalogOutNodeCarrier, RDFUNCSine);
//设置通道1的频率为1kHz
RDAnalogOutNodeFrequencySet(ftHandle, aoCh, RDAnalogOutNodeCarrier, 1000);
//设置通道1的偏置电压和幅值, offset:0V, amplitude:4V
RDAnalogOutNodeOffsetAmpSet(ftHandle, aoCh, RDAnalogOutNodeCarrier, 0, 4,
calArr);
//设置通道1的对称性
RDAnalogOutNodeSymmetrySet(ftHandle, aoCh, RDAnalogOutNodeCarrier, 50);
//设置通道1的初始相位为0
RDAnalogOutNodePhaseSet(ftHandle, aoCh, RDAnalogOutNodeCarrier, 0);

//启动模拟输出通道1
RDAnalogOutConfigure(ftHandle, aoCh, true);

//等待1秒至输出稳定
Sleep(1000);

//启动模拟输入
RDAnalogInConfigure(ftHandle, true);

//查询模拟输入状态直到状态为RDStateDone
printf("Waiting for triggered...\n");
do {
    RDAnalogInStatus(ftHandle, &sts);
} while (sts != RDStateDone);

//读取模拟输入通道1的结果
RDAnalogInRead(ftHandle, aiCh, cSamples, range, rgdSamples, calArr, &backsize);
printf("Scope ch1 data size: %d\n", backsize);
printf("Scope ch1 max. value: %fV\n", *std::max_element(rgdSamples, rgdSamples +
backsize));

printf("Done.\n");

//关闭模拟输出通道1
RDAnalogOutConfigure(ftHandle, aoCh, false);

//关闭模拟输入
RDAnalogInConfigure(ftHandle, false);

//关闭设备
RDdeviceClose(ftHandle);
}
```

3.3 使用 SDK 的 MATLAB API

已在 MATLAB R2024b 上经过测试。

I. 安装 MATLAB

按你的学校或老师提供的方法安装 MATLAB，安装时只需要 MATLAB 核心组件，不需要额外的工具箱（Toolboxes）；

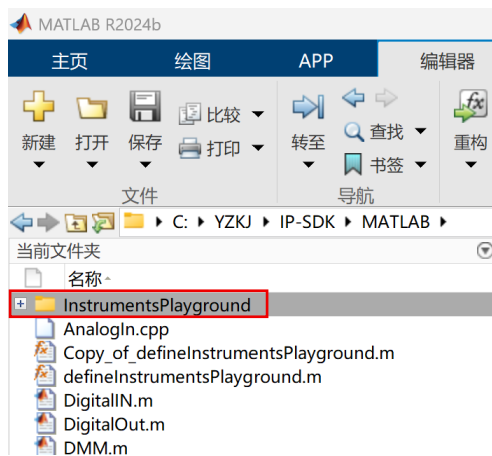
II. 打开 MATLAB，选择一个工作目录：

- 可以直接选择放置 SDK 的目录，即“C:\YZKJ\IP-SDK\MATLAB”；
- 也可以选择其他你喜欢的目录；



III. 将 IP-SDK 的库添加到 MATLAB 路径，有以下 2 种方法：

- 选项一：如果你的工作目录设置为 C:\YZKJ\IP-SDK\MATLAB，可以在左侧的文件浏览器中看到 InstrumentsPlayground 文件夹。右键点击该文件夹，选择“添加到路径→选定的文件夹和子文件夹”；



- 选项二：在 MATLAB 脚本的开头添加以下代码：

```
addpath("C:\YZKJ\IP-SDK\MATLAB\InstrumentsPlayground\")
```

IV. 运行代码

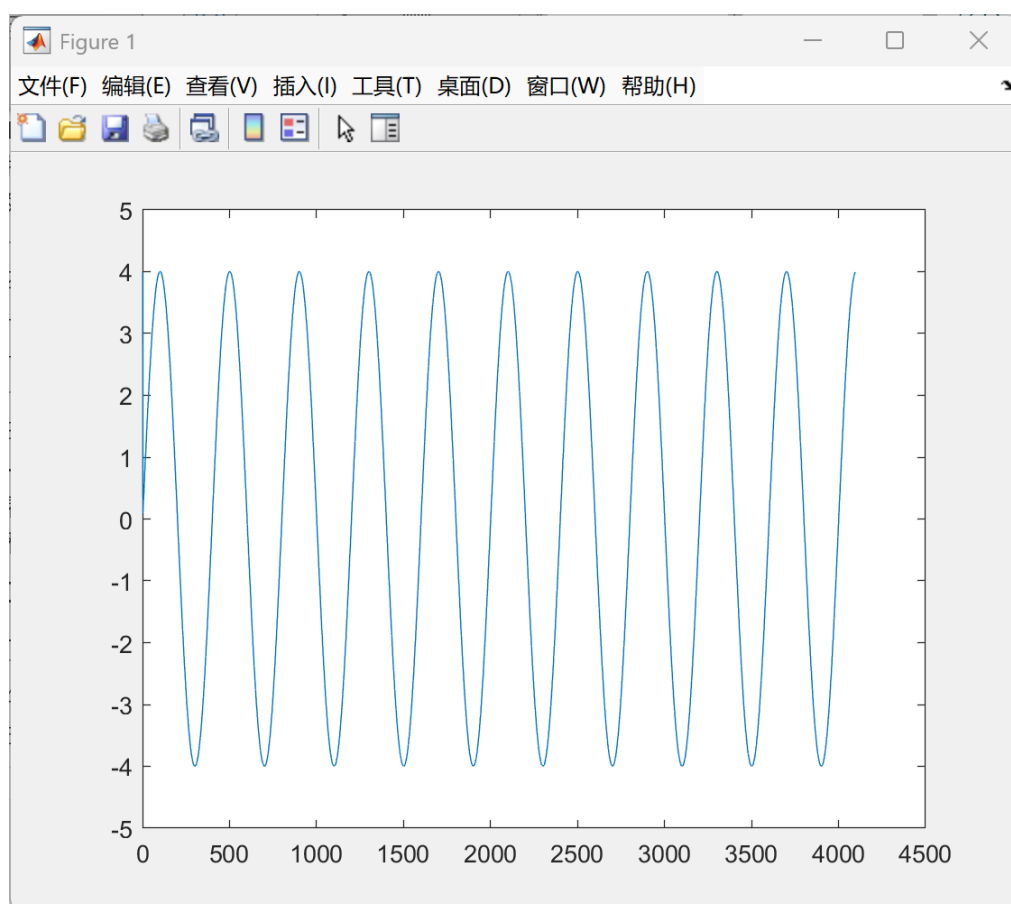
- 将示例代码拷贝到编辑器中，并保存为一个 MATLAB 脚本（.m 文件）；
- 点击上方的运行或按 F5 运行脚本；



- 如果运行成功，会显示以下结果：

命令行窗口

```
>> test
Available devices:
#0: ,
#1: USB <-> Serial Converter, YZJ00770
ret=0
sts=2
backsize=4096
Done. ret=false
fx>>
```



MATLAB 示例代码

```
%addpath("C:\YZKJ\IP-SDK\MATLAB\InstrumentsPlayground\)
%查询可用设备并打开第一台包含“YZ”的设备
[ret,ver]=clib.InstrumentsPlayground.RDSDKVersion();
[ret,count]=clib.InstrumentsPlayground.RDEnumDeviceCount();

device_selected=false;
device_no=0;
disp("Available devices:")
if count>0
    for i=0:count-1
        i=int32(i);
        [ret,info,sn,handle]=clib.InstrumentsPlayground.RDEnumDeviceInfo(i);
        disp("#"+i+": "+info+" "+sn)
        if ~device_selected || contains(sn,"YZ")
            device_selected=true;
            device_no=i;
        end
    end
end

[ret,handle]=clib.InstrumentsPlayground.RDdeviceOpen(device_no);

%读取校准值
[ret,calarray]=clib.InstrumentsPlayground.RDCalibrationRead(handle);

%模拟输入设置
aich=0;
range=5;
ret=ret|clib.InstrumentsPlayground.RDAnalogInChannelEnableSet(handle,aich,true);
ret=ret|clib.InstrumentsPlayground.RDAnalogInChannelRangeSet(handle,aich,range);
ret=ret|clib.InstrumentsPlayground.RDAnalogInFrequencySet(handle,400000);
ret=ret|clib.InstrumentsPlayground.RDAnalogInBufferSizeSet(handle,4096);

%模拟输入触发设置
ret=ret|clib.InstrumentsPlayground.RDAnalogInTriggerSourceSet(handle,1); %0:none,1:ch1
ret=ret|clib.InstrumentsPlayground.RDAnalogInTriggerTypeSet(handle,0); %0:edge
ret=ret|clib.InstrumentsPlayground.RDAnalogInTriggerLevelSet(handle,0,range,aich,calarray);
ret=ret|clib.InstrumentsPlayground.RDAnalogInTriggerConditionSet(handle,0);%0:rising_edge
ret=ret|clib.InstrumentsPlayground.RDAnalogInTriggerAutoTimeoutSet(handle,0);%0:forever,N>0:
waits_N(s)
```



```
%模拟输出设置
aoch=0;
node=0;
ret=ret|clib.InstrumentsPlayground.RDAnalogOutNodeEnableSet(handle,aoch,node,true);
ret=ret|clib.InstrumentsPlayground.RDAnalogOutNodeFunctionSet(handle,aoch,node,1);%0:DC,1:
Sine
ret=ret|clib.InstrumentsPlayground.RDAnalogOutNodeFrequencySet(handle,aoch,node,1000);
ret=ret|clib.InstrumentsPlayground.RDAnalogOutNodeOffsetAmpSet(handle,aoch,node,0,4,calarray);
ret=ret|clib.InstrumentsPlayground.RDAnalogOutNodeSymmetrySet(handle,aoch,node,50);
ret=ret|clib.InstrumentsPlayground.RDAnalogOutNodePhaseSet(handle,aoch,node,0);

%启动模拟输出和模拟输入
ret=ret|clib.InstrumentsPlayground.RDAnalogOutConfigure(handle,aoch,true);
pause(1)
ret=ret|clib.InstrumentsPlayground.RDAnalogInConfigure(handle,true);

%等待模拟输入就绪
sts=0;
for i=1:10
    [ret,sts]=clib.InstrumentsPlayground.RDAnalogInStatus(handle);
    if sts==2
        break
    end
end

disp("ret="+ret)
disp("sts="+sts)

%读取数据
[ret,buffer_ch1,backsize_ch1]=clib.InstrumentsPlayground.RDAnalogInRead(handle,aich,4096,range,calarray);

disp("backsize="+backsize_ch1)

%绘制数据
plot(1:backsize_ch1,buffer_ch1);

%关闭各模块和设备
ret=ret|clib.InstrumentsPlayground.RDAnalogOutConfigure(handle,aoch,false);
ret=ret|clib.InstrumentsPlayground.RDAnalogInConfigure(handle,false);
ret=ret|clib.InstrumentsPlayground.RDdeviceClose(handle);

disp("Done. ret="+ret)
```

4. 在树莓派的 Raspberry Pi OS 下使用 SDK

4.1 准备工作

1. 更换 apt 软件源以提高下载速度

依照 <https://mirrors.tuna.tsinghua.edu.cn/help/debian/> 中的操作说明（传统格式）更换软件源。

2. 安装 sqlite3

```
$ sudo apt update -y
$ sudo apt install sqlite3 libsqlite3-dev -y
```

如果在 apt update 时遇到类似错误：

E: could not get lock /var/lib/apt/lists/lock. It is held by process 1374 (packagekitd)

输入以下指令：

```
$ systemctl restart packagekit
```

3. 设置 FTDI 驱动

Source: <https://ftdichip.com/Driver/D2XX/Linux/ReadMe.txt>

前往 <https://ftdichip.com/drivers/d2xx-drivers/>，按硬件版本选择驱动：

- 树莓派 5：ARMv8
(<https://ftdichip.com/wp-content/uploads/2025/03/libftd2xx-linux-arm-v8-1.4.33.tgz>)
- 树莓派 4：ARMv7 hard-float
(<https://ftdichip.com/wp-content/uploads/2025/03/libftd2xx-linux-arm-v7-hf-1.4.33.tgz>)

然后执行以下指令：

```
# 下载你的 CPU 版本对应的驱动
$ wget https://ftdichip.com/wp-content/uploads/2025/03/libftd2xx-
linux-arm-v8-1.4.33.tgz
# 解压下载的文件
$ tar xfvz libftd2xx-linux-arm-v8-1.4.33.tgz
# 进入解压后的目录
$ cd linux-arm-v8
# 将库文件拷贝到系统目录下
$ sudo cp libftd2xx.so.1.4.33 /usr/local/lib
$ sudo cp libftd2xx-static.a /usr/local/lib
$ sudo ln -sf /usr/local/lib/libftd2xx.so.1.4.33 \
/usr/local/lib/libftd2xx.so
$ sudo ldconfig
```

4. 安装图形界面的仪器操场（可选）

获取仪器操场的 deb 安装包，并执行以下指令安装：

```
$ sudo dpkg -i InstrumentsPlayground_xxx.deb
```

4.2 使用 SDK 的 Python API

1. 检查是否已安装 Python

系统通常已经预装了 Python 3，在终端输入以下命令查看 Python 的版本：

```
$ python --version  
3.11.2
```

如果没有安装或想要升级到最新版本，输入以下指令：

```
$ sudo apt install python3 -y
```

2. 安装 Python 库

```
$ sudo apt install python3-numpy python3-matplotlib -y
```

3. 运行代码

树莓派的代码可在 [Windows 系统的 Python 示例代码](#) 上稍作修改，需要修改的部分为将以下代码中 SDK 的路径替换为当前 SDK 所在路径：

```
import sys  
# 将 IP-SDK (Python) 添加到系统环境变量 PATH 中  
sys.path.append('C:\YZKJ\IP-SDK\Python\src')
```

默认情况下应为：

```
sys.path.append('/home/[用户名]/IP-SDK\Python\src')
```

将示例代码拷贝到任意目录，然后运行代码：

```
$ cd [存放代码的目录]  
$ python [代码文件名].py
```

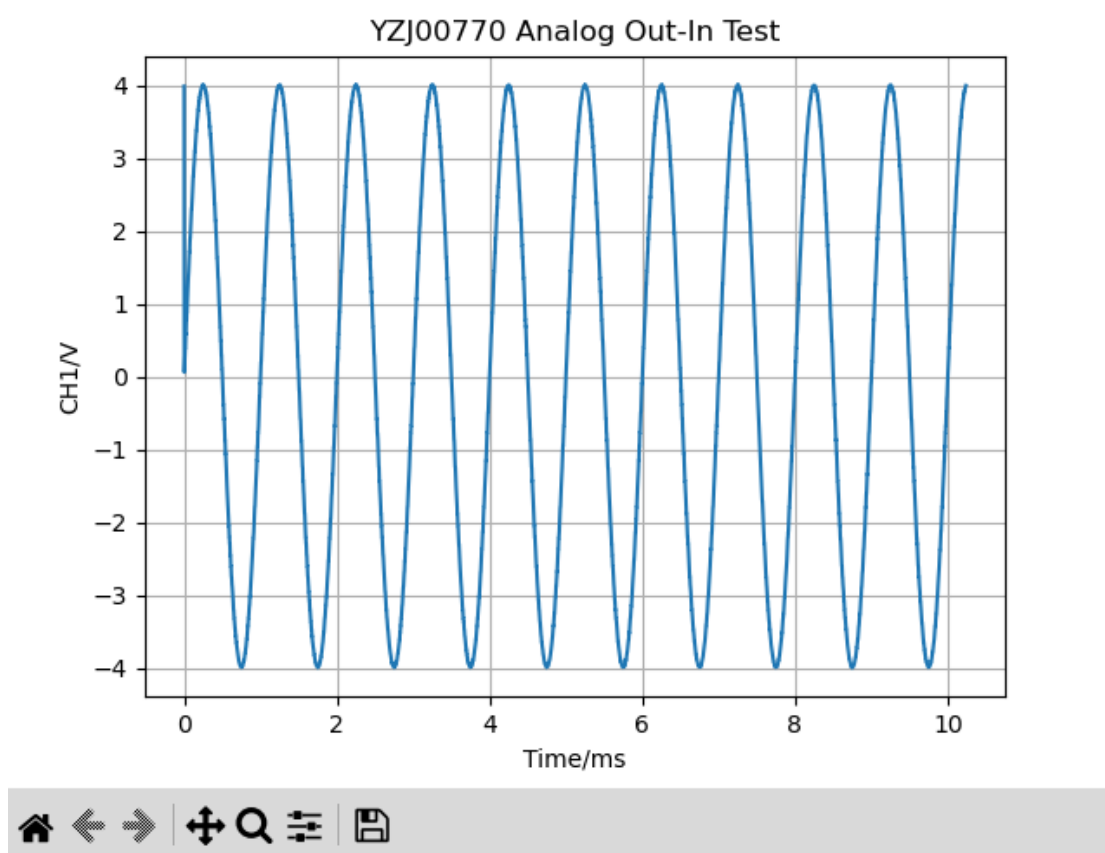
4. 运行成功后，应显示结果如下：

```

pi@raspberrypi: ~
File Edit Tabs Help
pi@raspberrypi:~ $ python AnalogOutInTest.py
You should change password at RDfunction.py at rmod Command to close FTDI VCP!
b'RDSDKVERSION3.0'
Available devices: [[b'USB <-> Serial Converter', b'YZJ00770']]
Unit SN: YZJ00770
open device success
Size of data read: c_int(4096)

```

Figure 1



4.3 使用 SDK 的 C/C++ API

1. 检查是否已安装 g++ 编译器

系统通常已经预装了 g++，在终端输入以下命令查看 g++ 的版本：

```
$ g++ --version
g++ (Debian 12.2.0-14+deb12u1) 12.2.0
...
```

如果没有安装或想要升级到最新版本，输入以下指令：

```
$ sudo apt install g++ -y
```

2. 将 IP 的动态库文件拷贝到系统下

```
$ sudo cp \
~/IPSDK/Python/lib/64/ARM64/libInstrumentsPlayground.so.1.0.0 \
/usr/local/lib
$ sudo ln -sf /usr/local/lib/libInstrumentsPlayground.so.1.0.0 \
/usr/local/lib/libInstrumentsPlayground.so
$ sudo ldconfig
```

3. 编译 C++ 源文件并运行

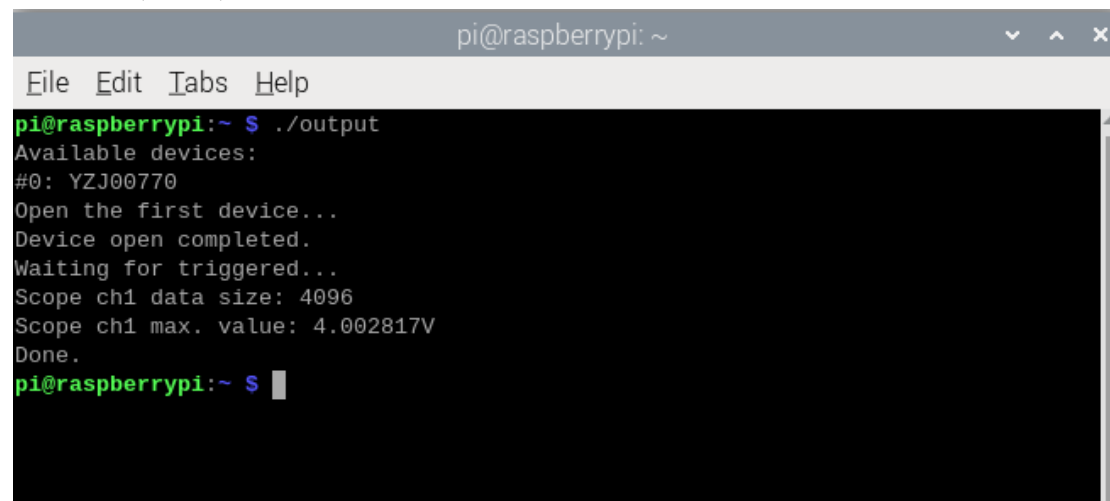
树莓派的代码可以在 [Windows 系统的 C/C++ 示例代码](#) 上稍作以下修改：将依赖 Windows API 的实现等待 1 秒功能的函数改为与 Linux 系统兼容的函数，即

- 将 <window.h> 替换为 <unistd.h>
- 将 Sleep(1000) 替换为 sleep(1)

将示例代码拷贝到 source.cpp 中，并在编译时使用 -I 指定头文件目录，以及 -l 指定动态链接库，然后运行编译后的文件：

```
$ g++ source.cpp -o output -I~/IP-SDK/C -lInstrumentsPlayground
$ ./output
```

4. 运行成功后，应显示结果如下：



```
pi@raspberrypi: ~
File Edit Tabs Help
pi@raspberrypi:~ $ ./output
Available devices:
#0: YZJ00770
Open the first device...
Device open completed.
Waiting for triggered...
Scope ch1 data size: 4096
Scope ch1 max. value: 4.002817V
Done.
pi@raspberrypi:~ $
```